

Aha-Secret

Owner: AHA-OIDA
Reviewer: whotwagner
Contributors:
Date Generated: Mon Jul 27 2026

Executive Summary

High level system description

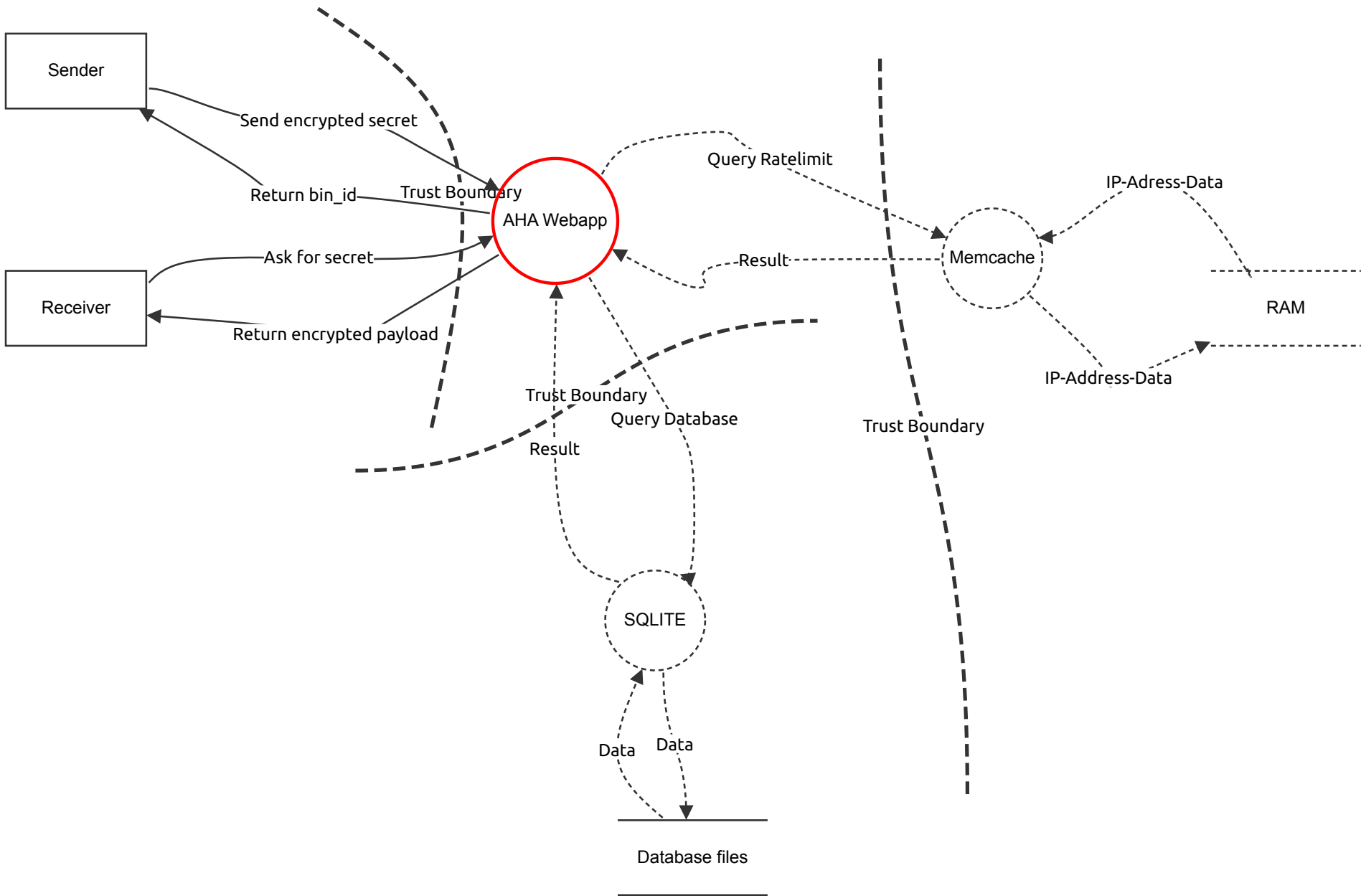
aha-secret allows you to store a secret message encrypted. Only the generated link can decrypt the message again. The message is encrypted by the sender's browser and decrypted by the recipient's browser. Secrets are deleted after the first reveal by default. Senders can instead allow repeated reveals for a retention time of up to one hour; those secrets remain available until scheduled expiry cleanup.

Summary

Total Threats	36
Total Mitigated	34
Total Not Applicable	1
Total Open	1
Open / Critical Severity	0
Open / High Severity	1
Open / Medium Severity	0
Open / Low Severity	0

STRIDE High Level

High level STRIDE-Diagram



STRIDE High Level

Send encrypted secret (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
15	Attacker fills up disk by sending data masses	Denial of service	Medium	Mitigated		Attacker posts very large amount of data in order to fill up the disk	* Ratelimit via Rack::Attack * Low Sizelimit for payload
16	Attacker fills up disk by sending many requests	Denial of service	Medium	Mitigated		Attacker sends many requests to fill up disk or exhaust other resources	Ratelimit via Rack::Attack
20	Attacker sends XSS-Payload to Reciever	Tampering	Medium	Mitigated		An attacker could send xss-payload to reciever to execute malicious javascript in the webbrowser of the reciever	* Using Content Security Policy using Rack::Protection * Textarea limits possible payloads for xss
21	Attacker injects SQL-commands	Tampering	Medium	Mitigated		Attacker injects sql commands	* Using well known ActiveRecord-gem that uses safe methods to interact with the database * Every record has an encrypted payload and the keys are never on the server
41	An eavesdropper can read the secret payload	Information disclosure	High	Mitigated		An eavesdropper can read the secret payload	* The payload is encrypted with AES256-GCM and the secret is never sent to the server * The webapp(crypto-api.js) only works with HTTPS or on local-installations
68	Encryption can be broken because of re-used Initial-Vectors	Information disclosure	Critical	Mitigated		Encryption can be broken if the same initial vector is used multiple times	Always use strong and cryptographic safe algorithm to generate random iv's
70	An attacker could send a victim malicious java-script-code	Tampering	Medium	N/A		An attacker could inject javascript to trick the victim to click on a link or show the victim malicious content(porn, etc...)	Strong CSP-Policy and CORS mitigates XSS-Attacks
75	Secret disclosure by keeping it in the textarea after reloading/back/forward	Information disclosure	Low	Mitigated		When user enters a secret and sends it, the browser might cache it and shows it again when comming back to the site. If this happens when other entities are watching, the secret could be compromised.	Automatically empty all textfields during initialisation/reloading of the page

Return bin_id (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
38	An eavesdropper could intercept the traffic and read out data	Information disclosure	Low	Mitigated		An eavesdropper could intercept the traffic and read out data. For example the bin_id.	The javascript crypto-library does only work if the backend runs on localhost or uses HTTPS
43	Adversary-in-the-Middle could tamper data	Tampering	Medium	Mitigated		Adversary-in-the-Middle could tamper data to produce xss or other malicious payloads	* HTTPS * HSTS
47	Adversary-in-the-Middle could tamper data to disrupt the service	Denial of service	Medium	Mitigated		Adversary-in-the-Middle could tamper data to disrupt the service	* HTTPS * HSTS

Return encrypted payload (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
42	An eavesdropper can read the secret payload	Information disclosure	High	Mitigated		An eavesdropper can read the secret payload	* The payload is encrypted with AES256-GCM and the secret is never sent to the server * The webapp(crypto-api.js) only works with HTTPS or on local-installations
44	Adversary-in-the-Middle could tamper data	Tampering	Medium	Mitigated		Adversary-in-the-Middle could tamper data to produce xss or other malicious payloads	* HTTPS * HSTS
48	Adversary-in-the-Middle could tamper data to disrupt the service	Denial of service	Medium	Mitigated		Adversary-in-the-Middle could tamper data to disrupt the service	* HTTPS * HSTS

Ask for secret (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
25	Enumerate possible payload-ids	Information disclosure	Medium	Mitigated		An attacker could try to enumerate payload identifiers and call the reveal endpoint in order to delete one-time secrets belonging to other people.	Using secure token of the ActiveRecord
40	An eavesdropper could intercept the traffic and read out data	Tampering	Low	Mitigated		An eavesdropper could intercept the traffic and read out data. For example the bin_id.	The javascript crypto-library does only work if the backend runs on localhost or uses HTTPS
49	Attacker disrupts service with flood of requests	Denial of service	Medium	Mitigated		Attacker disrupts service with flood of requests	Ratelimit via Rack::Attack
56	Receiver accidentally reveals and deletes a one-time secret	Tampering	Medium	Mitigated		A receiver can accidentally reveal and delete a one-time secret. For example, a sender creates a secret and reveals it instead of copying the link. Reusable secrets are not deleted on reveal but remain accessible to anyone with the link until scheduled expiry cleanup.	Opening the link does not retrieve the secret. Only PATCH /bins/:id/reveal fetches the encrypted payload. The sender can opt into reusable mode for group sharing, limited to a retention time of one hour.

Result (Data Flow) - *Out of Scope*

Reason for out of scope: Handled by ActiveRecord

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Query Database (Data Flow) - *Out of Scope*

Reason for out of scope: Handled by ActiveRecord

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Query Ratelimit (Data Flow) - *Out of Scope*

Reason for out of scope: Handled by Rack::Attack

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Result (Data Flow) - *Out of Scope*

Reason for out of scope:							
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Data (Data Flow) - *Out of Scope*

Reason for out of scope: Handled by sqlite							
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Data (Data Flow) - *Out of Scope*

Reason for out of scope:							
Number	Title	Type	Severity	Status	Score	Description	Mitigations

IP-Adress-Data (Data Flow) - *Out of Scope*

Reason for out of scope: Handled by Memcache							
Number	Title	Type	Severity	Status	Score	Description	Mitigations

IP-Address-Data (Data Flow) - *Out of Scope*

Reason for out of scope:							
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Database files (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
58	Dataleak	Information disclosure	High	Mitigated		The database-files could be accessed by unauthorized entities. For example by: * attackers hacked the system * administrators steal the data * old harddisk * aso...	* Data is encrypted on client side * Key is never passed on the server * Strong encryption algorithm is used on the client side * Data is decrypted on client side
59	Attacker on server-side could manipulate data of any user	Tampering	High	Mitigated		Attacker on server-side could manipulate data of any user	* Data is encrypted on client side * Key is never passed on the server * Strong encryption algorithm is used on the client side * Data is decrypted on client side

Number	Title	Type	Severity	Status	Score	Description	Mitigations
65	Decryption link leaked and an attacker got the secret	Information disclosure	High	Mitigated		AHA-Secret is not responsible for how the decryption-link is shared. If the link is somehow disclosed by the user and an unauthorized entity fetches the key, the secret will be compromised	Additional Password prevents this

AHA Webapp (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
10	Supply-Chain-Attack: Backdoored dependencies	Elevation of privilege	High	Mitigated		A backdoored dependency could compromise the server. An attacker could replace the javascript files to extract the secrets on the client side.	* Reduce dependencies: no nodejs
11	Vulnerable dependency allows code execution	Elevation of privilege	High	Mitigated		A vulnerability in any dependency or 3rd-party software opens a remote code execution vulnerability	* Minimum features -> less code -> keep attack vector small * Dependabot in Github checks for updates
55	Attacker on server-side could manipulate data of any user	Spoofing	High	Mitigated		Attacker on server-side could manipulate data of any user	* Data is encrypted on client side * Key is never passed on the server * Strong encryption algorithm is used on the client side * Data is decrypted on client side
51	Attacker on server-side could manipulate javascript to exfiltrate secrets	Tampering	High	Open		Attacker on server-side could manipulate javascript to exfiltrate secrets on the client-side.	
52	Decryption link leaked and an attacker got the secret	Repudiation	High	Mitigated		AHA-Secret is not responsible for how the decryption-link is shared. If the link is somehow disclosed by the user and an unauthorized entity fetches the secret, it must be detected.	* One-time mode lets the intended receiver notice that the link no longer works. Reusable mode cannot provide that signal before scheduled expiry cleanup. * Access logs * An additional password can prevent access with the link alone.
53	Dataleak	Information disclosure	High	Mitigated		The data could leak due to various reasons: * attackers hacked the system * administrators steal the data * old harddisk * aso...	* Data is encrypted on client side * Key is never passed on the server * Strong encryption algorithm is used on the client side * Data is decrypted on client side
54	DoS because of floods of requests	Denial of service	Medium	Mitigated		DoS because of floods of requests	Requst Ratelimit based on IP using Rack::Attack
62	Server-side attacker could guess encryption-key	Information disclosure	High	Mitigated		An attacker is able to get the database and tries to break the encryption	Client-side autogenerated cryptographic secure random keys prevent such attacks against AES256
64	Decryption link leaked and an attacker gets the secret	Information disclosure	High	Mitigated		AHA-Secret is not responsible for how the decryption-link is shared. If the link is somehow disclosed by the user and an unauthorized entity fetches the key, the secret will be compromised	Additional custom password prevents it
73	Weak custom password leads to info-leak	Information disclosure	Critical	Mitigated		The sender uses a weak custom password that can be bruteforced on the server. Such a scenario could expose the secrets on the server side	Use custom passwords only as additional passwords. The secret is still mainly encrypted with a strong random-key
74	Compromised memcache can elevate privileges via deserialisation vulnerability	Elevation of privilege	High	Mitigated		Per default dalli uses mashalling for deserialization. A compromised memcache could therefor lead to a remote code execution on the aha-secret server	aha-secret uses json for the memcache deserialisation

RAM (Store) - *Out of Scope*

Reason for out of scope:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Memcache (Process) - *Out of Scope*

Reason for out of scope:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

SQLITE (Process) - *Out of Scope*

Reason for out of scope:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Sender (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
28	An attacker forces an end user to execute unwanted actions on the webapp(CSRF)	Spoofing	Low	Mitigated		An attacker forces an end user to execute unwanted actions on the webapp(CSRF)	Using csrf-token and CORS via Rack::Protection
32	Attacker denies sending payload	Repudiation	Low	Mitigated		Attacker denies sending malicious payloads	Access-Logs

Receiver (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
29	An attacker forces an end user to execute unwanted actions on the webapp(CSRF)	Spoofing	Low	Mitigated		An attacker forces an end user to execute unwanted actions on the webapp(CSRF)	Using csrf-token and CORS via Rack::Protection
33	Attacker denies sending payloads	Repudiation	Low	Mitigated		Attacker denies sending malicious payloads	Access-Logs